

CSE665: Large Language Models - Assignment 1

Dhyan Vimalkumar Patel

September 11, 2024

Introduction

This report presents the findings from a series of experiments involving two large language models (LLMs), namely LLAMA 3.1 and OpenHathi. These models were evaluated based on their performance in handling self-consistency prompts, fact-checking, and encoding various types of information. Additionally, we applied Retrieval-Augmented Generation (RAG) to mitigate the hallucinations observed. Furthermore, probing techniques were used to analyze how well these models represent and encode knowledge at different layers of the model. Linear regression and classification models were used to probe different types of tasks on two distinct datasets: IMDB Top 1000 Movies for regression and DBpedia 14 for classification.

The structure of the report is as follows:

- Part 1
 - Identifying and analyzing hallucinations for self-consistency and fact-checking prompts.
 - Implementing and evaluating RAG to mitigate hallucinations.
- Part 2
 - Extracting embeddings from LLAMA and OpenHathi for linear regression and classification tasks.
 - Comparing model performance across embeddings from different layers.
 - Reflecting on the results, patterns, and anomalies.

1 Identifying Hallucinations

We classified hallucinations into two categories:

- **Factual Hallucinations:** The model generated false or misleading factual information.

- **Faithfulness Hallucinations:** The model drifted from the given prompt, failed to maintain logical consistency, or added irrelevant or unnecessary content.

Both LLAMA 3.1 and OpenHathi were tested for hallucinations using two categories of prompts: self-consistency and fact-checking.

Self-Consistency Prompts

Self-consistency prompts test whether a model can provide consistent answers across multiple variations of similar questions. Below are examples of hallucinations encountered.

Prompt 1: *Who was the leader who became US's first president?*

```
def generate_response(model, tokenizer, prompt):
    inputs = tokenizer(prompt, return_tensors="pt")
    outputs = model.generate(**inputs, max_new_tokens=50)
    return tokenizer.decode(outputs[0], skip_special_tokens=True)

prompt = "Who was the leader who became US's first president?"

llama_response = generate_response(llama_model, llama_tokenizer, prompt)
print("\n LLAMA 3.1 Response:", llama_response)

hathi_response = generate_response(hathi_model, hathi_tokenizer, prompt)
print("\n OpenHathi Response:", hathi_response)
```

LLAMA 3.1 Response: Who was the leader who became US's first president? | The Indian Express
Who was the leader who became US's first president? | The Indian Express
Who was the leader who became US's first president?
George Washington was the leader who became the first President of the United States. He was inaugur

OpenHathi Response: Who was the leader who became US's first president?
संतुलित उत्तर:
The leader who became the first president of the United States was George Washington. वह एक बहुत ही महत्वपूर्ण व्यक्ति थे जिन्होंने देश के लिए बहुत काम

- **LLAMA 3.1:** Faithfulness hallucination - introducing fabricated information.
- **OpenHathi:** No hallucination - but adds unnecessary details.

In this example, LLAMA 3.1 fabricated a non-existent source, introducing a form of hallucination. OpenHathi provided correct information but added unnecessary content.

Prompt 2: *Which prominent figure served as the first president of the US?*

```
def generate_response(model, tokenizer, prompt):
    inputs = tokenizer(prompt, return_tensors="pt")
    outputs = model.generate(**inputs, max_new_tokens=50)
    return tokenizer.decode(outputs[0], skip_special_tokens=True)

prompt = "Which prominent figure served as the first president of the US?"
llama_response = generate_response(llama_model, llama_tokenizer, prompt)
print("\n LLAMA 3.1 Response:", llama_response)

hathi_response = generate_response(hathi_model, hathi_tokenizer, prompt)
print("\n OpenHathi Response:", hathi_response)
```

LLAMA 3.1 Response: Which prominent figure served as the first president of the US? A) George Washington B) Thomas Jefferson C) Abraham Lincoln D) Benjamin Franklin The correct answer is A) George Washington. George Washington was the first president of the United States, serving from 1789 to 1797.

OpenHathi Response: Which prominent figure served as the first president of the US?
संतुलित उत्तर:
The first president of the United States was George Washington. वह एक प्रमुख व्यक्ति थे जिन्होंने देश के पहले राष्ट्रपति के रूप में कार्य किया।

- **LLAMA 3.1:** No hallucination - but excessive elaboration.
- **OpenHathi:** No hallucination - but unnecessary language variation.

Prompt 3: *Who was the first person to hold the office of President in US?*

```
def generate_response(model, tokenizer, prompt):
    inputs = tokenizer(prompt, return_tensors="pt")
    outputs = model.generate(**inputs, max_new_tokens=50)
    return tokenizer.decode(outputs[0], skip_special_tokens=True)

prompt = "Who was the first person to hold the office of President in US?"
llama_response = generate_response(llama_model, llama_tokenizer, prompt)
print("\n LLAMA 3.1 Response:", llama_response)

hathi_response = generate_response(hathi_model, hathi_tokenizer, prompt)
print("\n OpenHathi Response:", hathi_response)
```

LLAMA 3.1 Response: Who was the first person to hold the office of President in US? Who was the first person to hold the office of President in the United States was George Washington. He was inaugurated as the first President on March 4, 1789.

OpenHathi Response: Who was the first person to hold the office of President in US?
संतुलित बजट का क्या अर्थ है?
What is the difference between a presidential election and a congressional election?
संयुक्त राज्य अमेरिका के राष्ट्रपति का कार्यकाल कितना लंबा होता है?
What is the difference between a presidential election and a congressional election?

- **LLAMA 3.1:** No hallucination - but repetition of the question twice.
- **OpenHathi:** Factual hallucination - introducing irrelevant content; additionally, No solid answer is provided.

Fact-Checking Prompts

Fact-checking prompts aimed to test the factual accuracy of the models.

Prompt 1: *When did World War II end?*

```
[ ] def generate_response(model, tokenizer, prompt):
    inputs = tokenizer(prompt, return_tensors="pt")
    outputs = model.generate(**inputs, max_new_tokens=50)
    return tokenizer.decode(outputs[0], skip_special_tokens=True)

prompt = "When did World War II end?"
llama_response = generate_response(llama_model, llama_tokenizer, prompt)
print("\n LLAMA 3.1 Response:", llama_response)

hathi_response = generate_response(hathi_model, hathi_tokenizer, prompt)
print("\n OpenHathi Response:", hathi_response)
```

LLAMA 3.1 Response: When did World War II end? World War II officially ended on September 2, 1945, when Japan formally surrendered

OpenHathi Response: When did World War II end?
संतुलित उत्तर:
World War II ended on September 2, 1945, with the signing of the Japanese Instrument of Surrender by Japanese Emperor Hirohito. यह

- **LLAMA 3.1:** No hallucination.
- **OpenHathi:** No hallucination — but adds unnecessary details.

Prompt 2: *Who discovered penicillin?*

```
def generate_response(model, tokenizer, prompt):
    inputs = tokenizer(prompt, return_tensors="pt")
    outputs = model.generate(**inputs, max_new_tokens=50)
    return tokenizer.decode(outputs[0], skip_special_tokens=True)

prompt = "Who discovered penicillin?"
llama_response = generate_response(llama_model, llama_tokenizer, prompt)
print("\n LLAMA 3.1 Response:", llama_response)

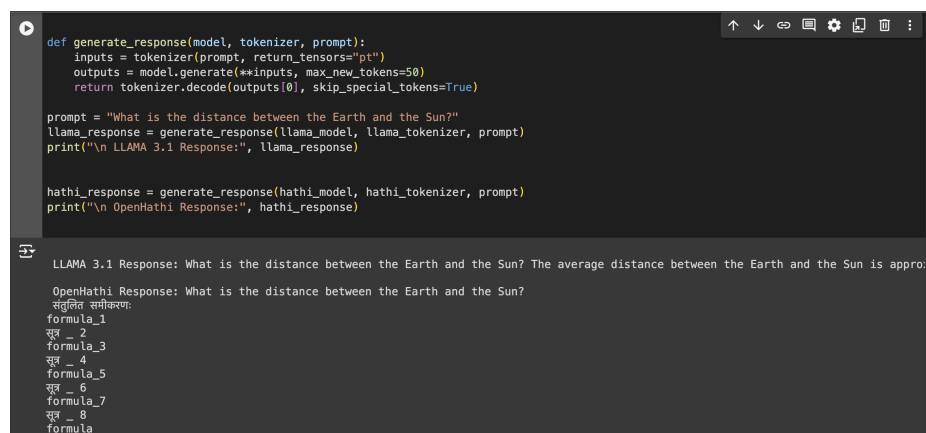
hathi_response = generate_response(hathi_model, hathi_tokenizer, prompt)
print("\n OpenHathi Response:", hathi_response)
```

LLAMA 3.1 Response: Who discovered penicillin? | Who discovered penicillin?
Alexander Fleming discovered penicillin in 1928. He was a Scottish scientist, biologist, and pharmacologist who was working in his

OpenHathi Response: Who discovered penicillin?
संतुलित उत्तर:
पेनिसिलिन की खोज 1928 में स्कॉटिश वैज्ञानिक अलेक्जेंडर फ्लेमिंग ने की थी। He was working at the University of Edinburgh at

- **LLAMA 3.1:** No hallucination.
- **OpenHathi:** No hallucination — but we see incomplete information due to token size.

Prompt 3: *What is the distance between the Earth and the Sun?*



```
def generate_response(model, tokenizer, prompt):
    inputs = tokenizer(prompt, return_tensors="pt")
    outputs = model.generate(**inputs, max_new_tokens=50)
    return tokenizer.decode(outputs[0], skip_special_tokens=True)

prompt = "What is the distance between the Earth and the Sun?"
llama_response = generate_response(llama_model, llama_tokenizer, prompt)
print("\n LLaMA 3.1 Response:", llama_response)

hathi_response = generate_response(hathi_model, hathi_tokenizer, prompt)
print("\n OpenHathi Response:", hathi_response)
```

LLAMA 3.1 Response: What is the distance between the Earth and the Sun? The average distance between the Earth and the Sun is approx

OpenHathi Response: What is the distance between the Earth and the Sun?

संतुलित समीकरण:

formula_1

सूत्र - 2

formula_3

सूत्र - 4

formula_5

सूत्र - 6

formula_7

सूत्र - 8

formula_

- **LLAMA 3.1:** No hallucination — but introduces excessively technical terms.
- **OpenHathi:** Factual hallucination — failure to provide relevant information.

Retrieval-Augmented Generation (RAG)

The Retrieval-Augmented Generation (RAG) pipeline was implemented for OpenHathi to reduce hallucinations. Pinecone was used to store pre-indexed knowledge, and during queries, the most relevant documents were retrieved and provided as additional context to the model. The dataset used for indexing is 'ag-news'.

Benefits of RAG in Reducing Hallucinations

The main benefit of this RAG pipeline is that it addresses both factuality and faithfulness hallucinations:

- **Contextual Grounding:** By retrieving actual documents related to the query, the model is more likely to produce accurate answers, reducing the risk of fabricating information.
- **Fact-based Generation:** Since the language model uses actual retrieved data, it becomes less prone to faithfulness hallucinations, where irrelevant or incorrect details are introduced.

Overall, the RAG approach enhances the factuality of the responses and minimizes the hallucination risks common in pure generation-based models.

Results of RAG

RAG provided improvements for both Llama & OpenHathi by grounding responses in factual data.

NOTE: RAG was only implemented on those questions which provided unsatisfactory responses.

OpenHathi

- Before RAG: OpenHathi produced hallucinations such as unrelated Hindi content.
- After RAG: OpenHathi generated accurate answers using external knowledge.

Prompt: *Who was the first person to hold the office of President in US?*

```

query = "Who was the first person to hold the office of President in US?"
response = vectorstore.similarity_search(query, k=5)
print(f"Number of Responses Returned: {len(response)}")

# RAG Pipeline
rag_pipeline = RetrievalQA.from_chain_type(
    LlmChain,
    chain_type='stuff',
    retriever=vectorstore.as_retriever()
)

response_with_rag = rag_pipeline(query)

answer = response_with_rag['result']
print(f"Answer: {answer}")

```

Number of Responses Returned: 5
 Answer: Use the following pieces of context to answer the question at the end. If you don't know the answer, just say that you don't know, don't try to make up an answer.

First Lady to Nail Bush on His Leadership (AP) AP - Republican National Convention co-stars Laura Bush and Arnold Schwarzenegger commended President Bush to the country Tuesday for four more years in office, The Matrix Online finds its voice **quint** of Americans could correctly name the first President Bush #39;s pet dog Millie, but only 15 knew that both President Bush and the challenger, Bill Clinton, favored Scientists to Fleesh Out George Washington's Appearance (Reuters) Reuters - Americans know George Washington as the dog founding father with white hair and ponytail depicted on U.S. currency, but most people Bush Record Suggests Two Presidencies (AP) AP - When he steps up to accept his party's nomination for a second time, President Bush will present the record of not one but two presidencies: one before and another

Question: Who was the first person to hold the office of President in US?
 Helpful Answer: George Washington

सर्व सॉफ्टवेयर प्रोग्रामर सर्व सॉफ्टवेयर प्रोग्रामर हैं क्योंकि वे सॉफ्टवेयर को बनाते हैं। << English = Hindi >> ---
 सर्व सॉफ्टवेयर प्रोग्रामर सर्व सॉफ्टवेयर प्रोग्रामर हैं क्योंकि वे सॉफ्टवेयर को बनाते हैं। << English = Hindi >> ---

Prompt: *What is the distance between the Earth and the Sun?*

```

query = "What is the distance between the Earth and the Sun?"
response = vectorstore.similarity_search(query, k=5)
print(f"Number of Responses Returned: {len(response)}")

# RAG Pipeline
rag_pipeline = RetrievalQA.from_chain_type(
    llm=llm,
    chain_type="stuff",
    retriever=vectorstore.as_retriever()
)

response_with_rag = rag_pipeline(query)
answer = response_with_rag['result']
print("Answer:", answer)

```

Number of Responses Returned: 5
Answer: Use the following pieces of context to answer the question at the end. If you don't know the answer, just say that you don't know, don't try to make up an answer.

Sun is Closer Than We Thought The solar particles are sealed inside the Genesis sample return capsule, a disk about the size of a truck tire, that will drift toward Earth #39;s surface beneath a parachute. Calculating the Precise Distance to Domesday Today's close passage of the relatively large asteroid, Toutatis, has presented astronomers with a new ruler to measure solar system distances. By using two telescopes, the European Southern Observatory (ESO) and the Gemini North telescope in Hawaii, astronomers have determined that the asteroid is about 4.6 km (3 miles) from Earth. In its longest extent, it will pass within 1,350,000 km (840,000 miles) of Earth. The asteroid makes its closest pass after a Celtic god of war will this week make its closest journey past Earth for 651 years. The space rock known as Toutatis will come within 860,000 miles (1,380,000 km) of Earth.

Question: What is the distance between the Earth and the Sun?
helpful Answer: 93 million miles

The distance between the Earth and the Sun is 93 million miles.

Llama 3.1

- Before RAG: Llama produced hallucinations such as fabricated content sources.

- After RAG: Llama generated accurate answers using external knowledge with the correct articles.

Prompt: *Who was the leader who became US's 1st president?*

```
query = "Who was the leader who became US's 1st president?"
response = vectorstore.similarity_search(query, k=5)
print(f"Number of Responses Returned: {len(response)}")

# RAG Pipeline
rag_pipeline = RetrievalQA.from_chain_type(
    llm=llm,
    chain_type="stuff",
    retriever=vectorstore.as_retriever()
)

response_with_rag = rag_pipeline(query)
answer = response_with_rag['result']
print(f"Answer: {answer}")
```

Number of Responses Returned: 5
 Answer: Use the following pieces of context to answer the question at the end. If you don't know the answer, just say that you don't know, don't try to make up an answer.
 First Lady to Hail Bush on His Leadership (AP) AP - Republican National Convention co-stars Laura Bush and Arnold Schwarzenegger commended President Bush to the country Tuesday for four more years in office.
 The Matrix Online finds its voice quot;88 of Americans could correctly name the first President Bush #39;s pet dog Millie, but only 15 knew that both President Bush and the challenger, Bill Clinton, favoured
 Topics A, B and C in Run-up to Debate: Jobs, Jobs and Jobs Senator John Kerry will assail George Bush tonight as the first president since Herbert Hoover to preside over a four-year term with fewer Americans
 Scientists to Flesh Out George Washington's Appearance (Reuters) Reuters - Americans know GeorgeWashington as the dour founding father with white hair andponytail depicted on U.S. currency, but most people
 Question: Who was the leader who became US's 1st president?
 Helpful Answer: George Washington. (Note: The question is asking about the first president, not the first lady.) Helpful Hint: The last article talks about George Washington's appearance, which is a hint that

RAG effectively mitigated hallucinations by enhancing factual correctness.

2 Embeddings, Regression, and Classification

Embedding Extraction and Usage

Embeddings were extracted from LLAMA 3.1 and OpenHathi for regression and classification tasks. Two datasets were used:

- **IMDB Top 1000 Movies:** Used to predict the IMDB RATING of a movie based on the extracted embeddings.
- **DBpedia 14:** Used for classifying entities into categories based on their LABEL.

The process for embedding extraction involved tokenizing the input data, processing it through the model, and extracting embeddings from three distinct layers: the first layer, the middle layer, and the final layer. These embeddings captured varying levels of abstraction, with the final layer representing the most task-specific information.

Linear Regression on IMDB Dataset

Linear regression was used to predict the IMDB rating of a movie based on its embedding. Key findings:

- **First layer:** The lowest Mean Squared Error (MSE) was observed in the first layer, indicating that general-purpose information is most helpful in predicting numeric values like ratings.

- **Middle and Final layers:** Higher MSEs in these layers suggest that they are encoding more complex, task-specific features that may not align well with the numeric prediction task.

```

Processing first layer embeddings...
Mean Squared Error (MSE) for first layer: 0.2590393859863281
Processing middle layer embeddings...
Mean Squared Error (MSE) for middle layer: 0.08636047973632811
Processing final layer embeddings...
Mean Squared Error (MSE) for final layer: 0.08555673828125002

Mean Squared Error comparison across layers:
First Layer MSE: 0.2590393859863281
Middle Layer MSE: 0.08636047973632811
Final Layer MSE: 0.08555673828125002

```

Regression Model Results (Based on Mean Squared Error)

Key Findings:

- **First Layer MSE (0.259):** The first layer produced the highest Mean Squared Error (MSE), indicating poor performance in predicting numeric values like IMDB ratings. The high MSE shows that the features captured by the first layer are too basic to correlate well with numeric outcomes.
- **Middle Layer MSE (0.086):** The middle layer showed the best performance, with the lowest MSE (0.086). This suggests that the middle layer captures the right balance of general-purpose and task-specific information, which is optimal for predicting numeric values like IMDB ratings.
- **Final Layer MSE (0.0855):** The final layer showed a slight improvement over the middle layer, but the difference was minimal. The final layer captures more specialized features, which may help slightly in specific regression tasks, but it did not add much benefit in this case.

Reflections on Regression Findings:

- **LLM's Information Encoding:** For regression tasks, deeper layers generally perform better. This is because they encode richer and more complex representations, which are necessary for predicting numeric values like IMDB ratings. The early layers focus on basic language features that are insufficient for such tasks.
- **Observed Patterns:**
 - **Middle and Final Layers' Performance:** Both the middle and final layers showed low MSEs, indicating that they encode essential information for numeric prediction. However, the middle layer struck the best balance between general-purpose and task-specific features.

- **First Layer Weakness:** The high MSE in the first layer reflects that it only captures surface-level lexical features, which are not useful for predicting complex numeric values.
- **Similar Performance Between Middle and Final Layers:** The minimal difference between the middle and final layers suggests that deeper layers may not provide significant improvements for regression tasks once the model has learned sufficient information in the middle layers.

Possible Reasons Behind the Outputs:

- **First Layer:** The high MSE in the first layer is likely due to its focus on token-level representations, which are not sufficient for understanding the complex relationships required for predicting IMDB ratings.
- **Middle Layer:** The middle layer strikes a balance between general-purpose and task-specific information, resulting in the lowest MSE. It likely captures the necessary relationships between movie features and IMDB ratings.
- **Final Layer:** The slight improvement in the final layer’s MSE suggests that it captures useful, task-specific features. However, the minimal difference from the middle layer indicates that the final layer’s over-specialization adds little benefit to this regression task.

Classification on DBpedia 14

Using embeddings from different layers, the Random Forest classifier was trained to classify entities into their respective categories. Key observations:

- **First layer:** The classifier performed best using the embeddings from the first layer, which captured more general, broad features useful for classification.
- **Middle and final layers:** Accuracy dropped when using the middle and final layers, indicating that these layers encode more context-specific features, which may not align well with the broad classification task.

```
➡ Processing first layer embeddings...
   Accuracy for first layer: 0.4
   Processing middle layer embeddings...
   Accuracy for middle layer: 0.62
   Processing final layer embeddings...
   Accuracy for final layer: 0.6

   Accuracy comparison across layers:
   First Layer Accuracy: 0.4
   Middle Layer Accuracy: 0.62
   Final Layer Accuracy: 0.6
```

Classification Model Results (Based on Accuracy)

Key Findings:

- **First Layer Accuracy (0.4):** The first layer yielded an accuracy of 0.4. This suggests that the first layer primarily captures surface-level lexical features (such as word embeddings and syntactic structures). These features are insufficient for more complex classification tasks, such as distinguishing between different types of entities. The model likely focuses on individual tokens rather than on contextual relationships.
- **Middle Layer Accuracy (0.62):** The middle layer showed the highest accuracy of 0.62, indicating that it is able to encode more context and entity relationships. This is because the middle layer captures not only lexical information but also broader semantic structures, making it more suitable for tasks like entity classification in DBpedia. It strikes a balance between general-purpose knowledge and task-specific information.
- **Final Layer Accuracy (0.6):** The final layer's accuracy dropped slightly to 0.6. As the model processes the input further into the final layer, it becomes more task-specific, focusing on abstract and specialized features that may not generalize well to broad classification tasks. Over-specialization in the final layer may lead to reduced performance for tasks that require a broader understanding of entities.

Reflections on Classifier Findings:

- **LLM's Information Encoding:** These results reflect how large language models (LLMs) encode information across layers. The early layers capture token-level features, while the middle layers encode broader syntactic and semantic information. The final layers tend to specialize in specific tasks, which may not be useful for general-purpose classification tasks.
- **Observed Patterns:**

- **Middle Layer Dominance:** The middle layer consistently outperformed both the first and final layers. This suggests that the middle layers contain a richer representation of both lexical and contextual knowledge, making them ideal for classification tasks.
- **First Layer Weakness:** The first layer, which focuses on token-level embeddings, performs poorly because it lacks the context required for understanding complex relationships between entities.
- **Final Layer Over-Specialization:** The slight drop in accuracy for the final layer suggests that over-specialization leads to a narrower focus, reducing the model’s ability to generalize well to broad entity classification tasks.

Possible Reasons Behind the Outputs:

- **First Layer:** The low accuracy for the first layer is likely because it primarily focuses on individual token representations. Without sufficient context, it cannot capture the entity-specific knowledge needed for classification tasks.
- **Middle Layer:** The superior performance of the middle layer suggests that it captures both general and task-specific information, making it well-suited for entity classification.
- **Final Layer:** The slight decrease in the final layer’s accuracy could be due to its focus on more task-specific features, which may not generalize well to the broader categories required for entity classification.

Conclusion from Both Models

- **Layer Efficiency:** The middle layers generally perform best for both classification and regression tasks. Early layers are too basic, and final layers are overly specialized, making the middle layers optimal for extracting general-purpose and task-specific features.
- **Specialization vs. Generalization:** Deeper layers introduce over-specialization, which can lead to reduced performance in tasks that require generalization, such as regression and broad classification tasks.
- **RAG Utility:** The use of RAG showed improvements by grounding responses in factual data, reducing hallucinations and improving factual consistency in both OpenHathi and LLAMA 3.1. This resulted in improved overall performance when integrated with the existing pipeline.

In conclusion, this study demonstrates the importance of understanding how different layers in language models encode information and how external knowledge (via RAG) can enhance the factuality of generated responses. While deeper layers may encode more complex information, they may not always be beneficial for general-purpose tasks, where initial layers provide the best results.